

Launch Readiness — The First 1,000 Clients

50pick · 50pick.tz · 3 September 2026 · production commit 78c3876e . All figures measured on the live system today.

1 · What we pay now

Service	Per month	What it is for
Railway — 50pick hosting	\$23.57	The application, database and cache
Railway — five other projects	\$8.24	Amanat Watan, Intercontinental Glass, LMC, two others
Anthropic — AI	\$50.75	Writing markets and auto-settling results
Cloudflare R2	~\$1.00	Customer ID documents and backups
Postmark email	\$0.00	Inside the free allowance — 20 emails sent
Sentry	\$0.00	Free tier
SMS	\$0.00	Not sending anything — see item 1 below
Twelve Data price feed	not measured	Billed outside Railway — needs confirming
Total today	≈ \$84	

Railway's last four bills were \$5.00, \$11.15, **\$25.42** and \$17.38 so far this period. The cost is rising because the platform is doing more work, not because anything is wrong.

Two facts that decide where the money goes.

The AI bill is bigger than the hosting bill — \$50.75 against \$31.81 — and **99.98% of it does not depend on customer numbers**. It is the cost of creating and settling markets on a clock. Going from 104 customers to 1,000 will barely change it.

Memory is 87% of the 50pick hosting bill — \$20.60 of \$23.57. The application uses almost no processing power and holds 4.7 GB of memory. Capping that is the cheapest saving we have.

2 · What we will pay with 1,000 clients

Sized for 1,000 registered customers with 100–200 online at once.

Service	Now	At 1,000	Reason it changes
Railway — 50pick	\$23.57	\$56 – 107	More visitors, more memory, more traffic
Railway — other projects	\$8.24	\$8 – 10	Unchanged
Anthropic — AI	\$50.75	\$55 – 110	Market creation is flat. Only customer chat grows.
SMS sign-up codes	\$0	\$35 – 45	New cost — about 4,000 messages a month
Postmark email	\$0	\$0 – 15	Free to 100 messages a month
Cloudflare R2	~\$1	\$1 – 3	More ID documents stored

Total	≈ \$84	\$155 – 290	
-------	--------	-------------	--

Budget \$200 per month. Two lines can run away and both are controllable: the AI bill, set by how often we generate markets, and the memory bill, set by a configuration limit.

3 · What must be done before we open, and why

	Item	Cost	Reason
●	Turn on SMS	\$35–45/mo	No member of the public can register today. Sign-up requires a code by text message; the platform is set to write those codes to a log file instead of sending them. It has never sent one. Needs a signed Selcom contract and a TCRA sender ID — a commercial step, not engineering.
●	Two-factor on the admin console	\$0	Nine administrator accounts control customer money and are protected by a password only. Second-factor was switched off and was meant to be restored on 31 July.
●	Health check on deployments	\$0	Railway is not checking whether a new version works before sending customers to it. A broken release goes live unchallenged. The status page also reports "healthy" while the database is down, so it must be fixed before it can be trusted.
●	Continuous database backup	~\$2/mo	We back up once a night. A failure at the wrong hour loses up to 24 hours of bets, deposits and payouts . Railway provides point-in-time recovery on our existing plan; it is not switched on.
●	Alerts that arrive by themselves	\$0	In August the nightly backup failed eleven nights running . The warning was displayed correctly for ten of them and nobody saw it. Today the same failure would repeat the same way.
●	Updates without downtime	\$0	Every release briefly takes the site down and cancels any bet being placed at that moment. At five customers a day nobody notices. At a thousand, everybody does.
●	Payouts on large markets	\$0	Winners are paid one at a time inside a thirty-second limit. Measured, that runs out at about 660 winners . Our biggest market ever had 29 bets, so it has never been reached — one popular football match with a thousand customers will pass it, and the market then freezes for betting and cashing out.
●	Cache the busy screens	\$2–4/mo	Nothing is cached. Every page is rebuilt from the database on every view, and the main screens refresh themselves every 15–20 seconds. At 200 customers online that is roughly 400 database queries a second to redraw screens that have barely changed.
●	Move servers to Amsterdam	\$0	Servers are in California; customers are in Tanzania. The server answers in 31 milliseconds but a customer waits about half a second. That is distance, not slow software. Moving halves it and costs nothing. Requires one short planned outage.

●	Enlarge the database disk	\$0 now	The disk is 897 MB of 5 GB and grows 14 MB a day — about six months left. The compliance record inside it must be kept seven years by law and cannot be deleted. Railway charges for space used, not reserved, so enlarging it costs nothing until it is filled.
●	Recovery procedure and rehearsal	\$0	There is no written procedure for restoring the platform, and no measured recovery time. The restore tool was rehearsed once in July against a database a tenth of today's size.

Two of these are decisions, not engineering. Turning on SMS needs a signed contract. Two-factor needs an administrator to enrol an authenticator app. Everything else is a few days of work and buys no new infrastructure.

4 • Decisions taken

Question	Decision
How many customers online at once?	Size for 100–200 at peak
Selcom SMS contract	In progress — staged as a reversible switch, ready the day it lands
Server location	Move to Amsterdam in an announced maintenance window
Run several copies of the application?	No. One container serves 200 online comfortably once caching is in. Running several would first require rebuilding emergency controls that currently reach only one copy.

Part Two · Technical detail

MEASURED 3 SEPTEMBER 2026

Architecture as deployed

Players (Tanzania)
| no CDN, no firewall, every response uncached
v
Railway edge network ----> ORIGIN: us-west2 (California) -- all three services
|
+-- 50pick Next.js 15, RAILPACK 1 container
| no health check, no deploy overlap, no graceful shutdown
+-- PostgreSQL 18.6 1 container, 5 GB disk (897 MB used)
+-- Redis 1 container, connected, cross-container

External services
Selcom deposits and withdrawals LIVE (apigw.selcommobile.com/v1)
Postmark transactional email LIVE, 20 sent, 0 failures
Cloudflare R2 . KYC documents + backups LIVE
Sentry error reporting LIVE, PII scrubbing proven
Anthropic market generation + settling ... LIVE
Twelve Data ... Up & Down price oracle LIVE
SMS one-time codes NOT DELIVERING (provider = console)

31 ms
server p50 (p95 136 ms)

0
5xx errors in 500 requests

4.68 GB
average memory (peak 8.86)

14 MB
disk growth per day

Measured platform state

Item	Measured value
Railway plan	Pro — 42 replicas allowed, volumes to 1 TB, 10,000 connections per host
Deploy configuration	numReplicas 1, healthcheckPath null, overlapSeconds null, drainingSeconds null on all three services
PostgreSQL	18.6 · max_connections 500 (497 usable) · 49 in use · archive_mode off — no point-in-time recovery
Database size	450 MB. AuditLog is 279 MB of it — 7-year retention, cryptographically chained, cannot be deleted
Disk growth	+100 MB over 7 days. 89% is automated market and round activity, not customers
Connection pool	40 per container. 6 containers = 480 during a rolling deploy, inside the 497 limit
Redis	Connected, subscribed, cross-container. Fail-open behind a circuit breaker; never on the bet path
Real activity today	104 registered · 2–5 betting customers per day · largest market ever: 29 bets from 12 people

Page timing	~510 ms to first byte, <i>identical on every route</i> — the signature of distance, not slow queries
-------------	--

Two figures in our own internal notes were wrong, both in our favour. The code and documentation state that the database allows 100 simultaneous connections and that Redis is switched off in production. Measured live: the database allows **500**, and Redis is **connected and working across containers**. Several "we cannot scale" conclusions written earlier this year rested on those two stale figures and no longer hold.

Money safety — built, tested, and not to be rebuilt

- **Maintenance mode** pauses new bets and deposits while leaving withdrawals open, so customer funds are never trapped by an operational decision.
- **Payment kill switches** per mobile network, separately for money in and money out.
- **Deposits are settled from a signed re-query, not the incoming message**, exactly once, defended against a tampered amount, and recovered by a 15-second poller and a 5-minute sweep even if the provider's confirmation never arrives.
- **A dead Redis cannot break a bet** — proven by a test that points the platform at a dead port and asserts betting still succeeds.
- **A retry without an idempotency key is refused**, so a retry can never become a second bet.
- **200 simultaneous bets on one market: zero shillings lost or created.**
- **The Up & Down safety net is independent of the game's own off switch** — turning the game off must not disable the thing that returns money already staked in it.
- **Backups are verified by actually restoring them** nightly into a scratch database and re-checking the money totals and the audit chain — not by confirming a file was written.

Recovery position

Question	Today	After the work
Data loss window	Up to 24 hours — one backup a night, nothing between	Minutes
Time to restore	Never measured. Rehearsed once, July, against a database a tenth of today's size	Measured and recorded
Written procedure	None	One-page recovery runbook
Would we be told?	Errors yes. Backup failure no.	Alerts that arrive unprompted
Trade while degraded?	Yes — maintenance mode works today	Unchanged, and written into the procedure

Order of work, and how each is proved

#	Step	Proof it is done
1	Turn on Selcom SMS	Register on a real handset; read the code off the phone
2	Enrol and enforce admin two-factor	Full sign-in with a real code before removing the override

3	Health check, deploy overlap, graceful shutdown	Break the database deliberately and watch the check go red
4	Enlarge the database disk	New size visible; site stays up throughout
5	Cache the busy screens	Re-measure page timing and query rate before and after
6	Move to Amsterdam	Re-run the timing test against today's 510 ms baseline
7	Batch large-market payouts	Settle a test market with over 1,000 winners and time it
8	Turn on point-in-time recovery	Perform a real restore into a scratch database
9	Alerts that arrive by themselves	Break the backup on purpose; confirm the alert lands
10	Rehearse recovery; write the runbook	Record the actual wall-clock recovery time

Step 3 is already built and proved on branch `launch-1k-readiness`: the status page now returns "not ready" when the database is unreachable, tested both ways — red with a dead database, green without one. It is deliberately not yet on the live branch.

Prepared 3 September 2026 against production commit `78c3876e`. Sources: Railway billing API, Railway metrics API, the production PostgreSQL database, the platform's own AI spend ledger, and timed requests to `50pick.tz`. The only unmeasured figure is the Twelve Data subscription, billed outside Railway. Rebuild with `node docs/runbooks/mkpdf.mjs ../launch-1k.html ../50pick-launch-1k.pdf`.